Sorting

Why Sort?

Whenever data is collected, sooner or later we need to sort it. Names in a contact list, prices on a product page, scores on a leaderboard, items in a to-do list — sorting is everywhere. After sorting, searching the data also becomes much faster (binary search, for example, only works on sorted data). Sorting algorithms differ in several important ways:

- Time complexity — how the running time grows with the size of the input.
- Memory usage — do we sort in place, or do we need extra space?
- Recursive vs. iterative — some algorithms are most naturally expressed with recursion (like quick sort), others with simple loops.
- Comparison-based vs. not — most algorithms compare elements directly (bubble sort, insertion sort, etc.), but a few (bucket sort, pigeonhole sort) use other techniques.

1. Bubble Sort

The idea of bubble sort is simple: walk through the list comparing adjacent pairs of elements; whenever two elements are out of order, swap them. The largest element "bubbles" up to the end of the list on each pass.

1.1. The Swap

Every comparison-based sort needs to swap two elements. We use a temporary variable so we don't overwrite a value before we've used it:

Swap of unordered_list[j] and unordered_list[j+1]:

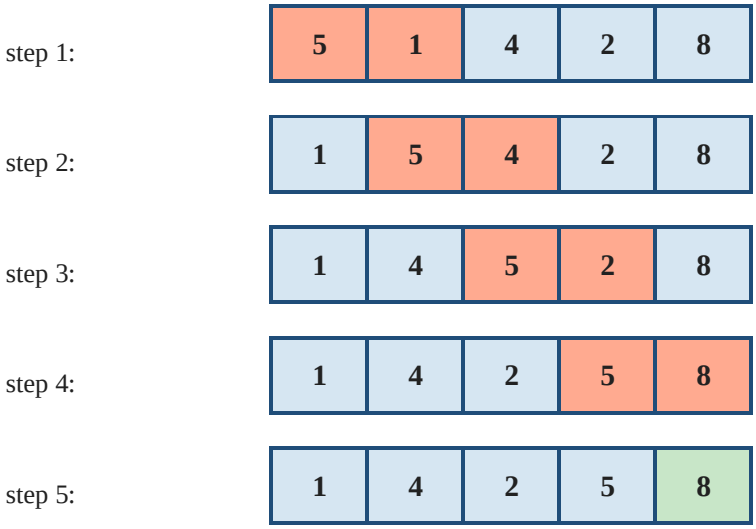
temp = list[j] → list[j] = list[j+1] → list[j+1] = temp

Swap with a temporary variable: 5 and 2 trade places.

```
temp = unordered_list[j]
unordered_list[j] = unordered_list[j+1]
unordered_list[j+1] = temp
```

1.2. One Full Pass

Look at what one pass of bubble sort does to the list [5, 1, 4, 2, 8]:



Adjacent pairs are compared and swapped if out of order. After one full pass the largest element (8) is in its final spot.

After a single pass the largest element (8) ends up in its final position at the rightmost slot.

To sort the entire list we have to repeat the pass n-1 times. Two nested loops do the job:

```
def bubble_sort(unordered_list):
    iteration_number = len(unordered_list) - 1
    for i in range(iteration_number):
       for j in range(iteration_number):
          if unordered_list[j] > unordered_list[j+1]:
             temp = unordered_list[j]
             unordered_list[j] = unordered_list[j+1]
             unordered_list[j+1] = temp
   return unordered_list
```

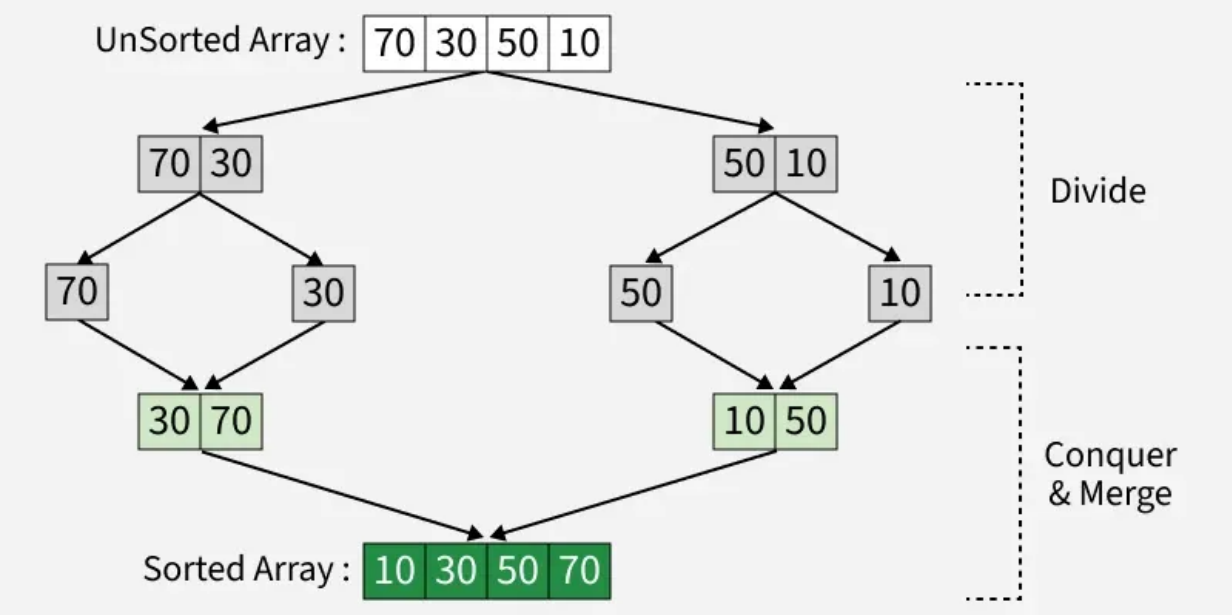
Example:

```
>>> bubble_sort([3, 2, 1])
[1, 2, 3]
```

Complexity: worst case $O(n^2)$, best case $O(n)$. Bubble sort is too slow for large lists, but it is fine for small ones and is often the very first sort taught because it is so easy to understand. A common optimization is to break out of the algorithm as soon as a full inner pass produces no swaps — that means the list is already sorted.

2. Merge Sort

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the Divide and Conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.



Here's a step-by-step explanation of how merge sort works:

1. **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
2. **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
3. **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Let's sort the array or list **[38, 27, 43, 10]** using Merge Sort

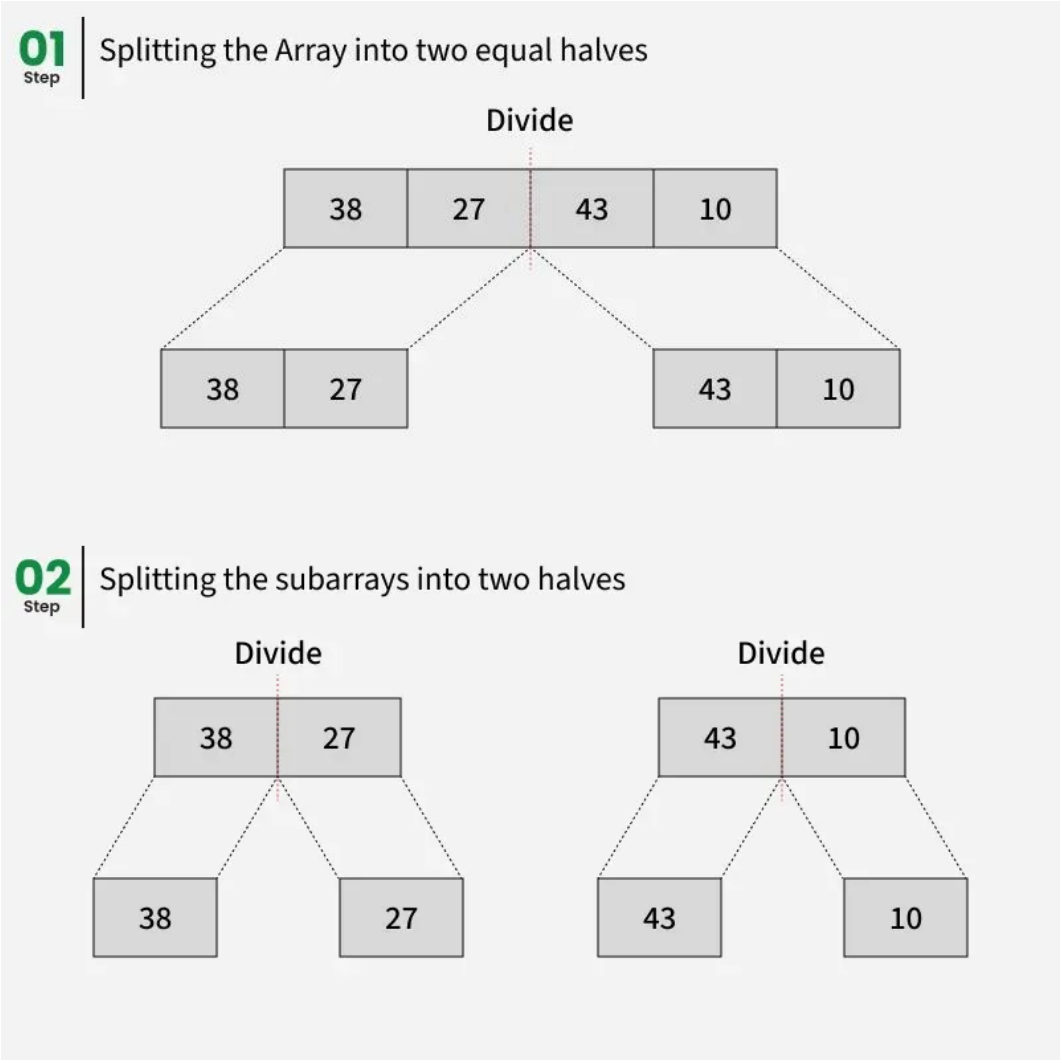
Divide:

- **[38, 27, 43, 10]** is divided into **[38, 27]** and **[43, 10]** .
- **[38, 27]** is divided into **[38]** and **[27]** .
- **[43, 10]** is divided into **[43]** and **[10]** .

Conquer:

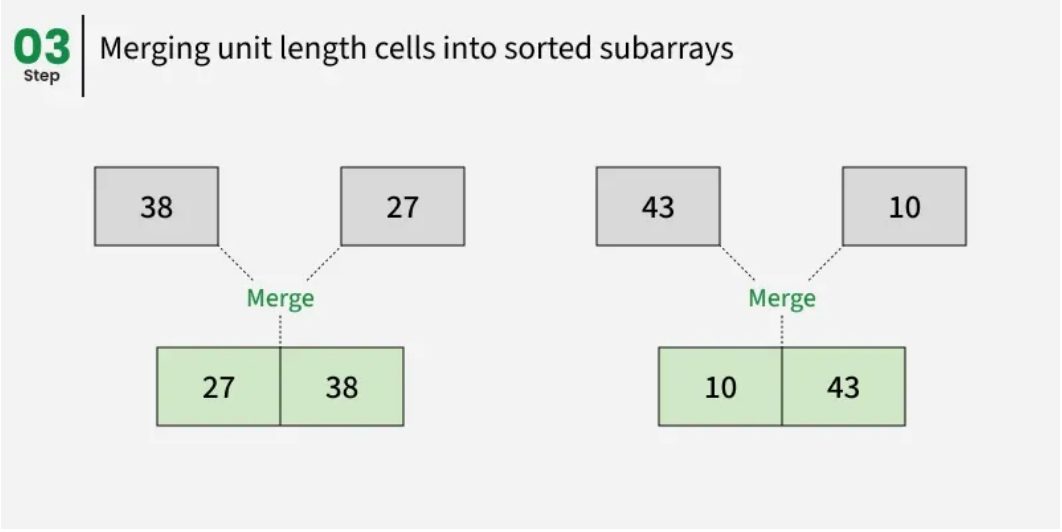
- **[38]** is already sorted.
- **[27]** is already sorted.
- **[43]** is already sorted.

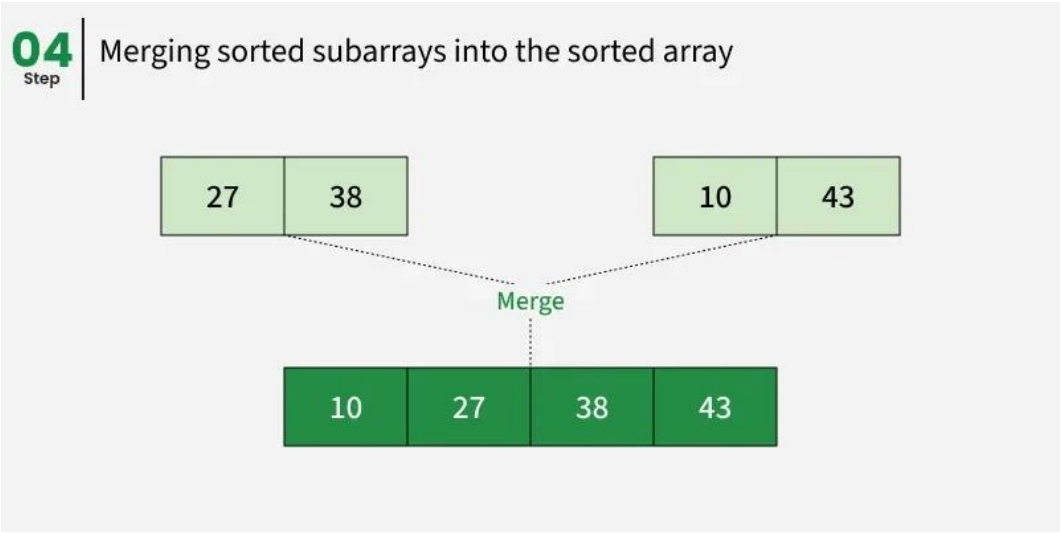
- **[10]** is already sorted.



Merge:

- Merge **[38]** and **[27]** to get **[27, 38]** .
- Merge **[43]** and **[10]** to get **[10,43]** .
- Merge **[27, 38]** and **[10,43]** to get the final sorted list **[10, 27, 38, 43]**





Therefore, the sorted list is **[10, 27, 38, 43]** .

```
def merge(arr, left, mid, right):
    n1 = mid - left + 1
    n2 = right - mid

    # Create temp arrays
    L = [0] * n1
    R = [0] * n2

    # Copy data to temp arrays L[] and R[]
    for i in range(n1):
        L[i] = arr[left + i]
    for j in range(n2):
        R[j] = arr[mid + 1 + j]

    i = 0
    j = 0
    k = left

    # Merge the temp arrays back into arr[left..right]
    while i < n1 and j < n2:
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1

    # Copy the remaining elements of L[],if there are any
    while i < n1:
        arr[k] = L[i]
        i += 1
        k += 1
```

```
    # Copy the remaining elements of R[],if there are any
    while j < n2:
        arr[k] = R[j]
        j += 1
        k += 1

def mergeSort(arr, left, right):
    if left < right:
        mid = (left + right) // 2

        mergeSort(arr, left, mid)
        mergeSort(arr, mid + 1, right)
        merge(arr, left, mid, right)

# Driver code
if __name__ == "__main__":
    arr = [38, 27, 43, 10]

    mergeSort(arr, 0, len(arr) - 1)
    for i in arr:
        print(i, end=" ")
    print()
```

Output

```
10 27 38 43
```

Recurrence Relation of Merge Sort

The recurrence relation of merge sort is:

$$T(n)=\{\Theta(1)\text{if } n=1 2T(n2)+\Theta(n)\text{if } n>1 T(n)=\{\Theta(1)2T(2n)+\Theta(n)\text{if } n=1\text{if } n>1$$

- T(n) Represents the total time time taken by the algorithm to sort an array of size n.
- 2T(n/2) represents time taken by the algorithm to recursively sort the two halves of the array. Since each half has n/2 elements, we have two recursive calls with input size as (n/2).
- O(n) represents the time taken to merge the two sorted halves

Complexity Analysis of Merge Sort

Time Complexity:

- **Best Case:** O(n log n), When the array is already sorted or nearly sorted.
- **Average Case:** O(n log n), When the array is randomly ordered.
- **Worst Case:** O(n log n), When the array is sorted in reverse order.
- **Auxiliary Space:** **O(n), Additional space is required for the temporary array used during** merging.

Applications of Merge Sort:

- Sorting large datasets
- External sorting (when the dataset is too large to fit in memory)
- Used to solve problems like Inversion counting, Count Smaller on Right & Surpasser Count
- Merge Sort and its variations are used in library methods of programming languages. Its variation TimSort is used in Python, Java Android and Swift. The main reason why it is preferred to sort non-primitive types is stability which is not there in QuickSort. Arrays.sort in Java uses QuickSort while Collections.sort uses MergeSort.
- It is a preferred algorithm for sorting Linked lists.
- It can be easily parallelized as we can independently sort subarrays and then merge.
- The merge function of merge sort to efficiently solve the problems like union and intersection of two sorted arrays.

Advantages and Disadvantages of Merge Sort

Advantages

- **Stability** : Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
- **Guaranteed worst-case performance:** Merge sort has a worst-case time complexity of **O(N logN)** , which means it performs well even on large datasets.
- **Simple to implement:** The divide-and-conquer approach is straightforward.
- **Naturally Parallel** : We independently merge subarrays that makes it suitable for parallel processing.

Disadvantages

- **Space complexity:** Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
- **Not in-place:** Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- Merge Sort is **Slower than QuickSort in general as** QuickSort is more cache friendly because it works in-place.